LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 2

Android Layout With Compose

Oleh:

Hamka Arifani NIM. 2410817110006

PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
APRIL 2026

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM PEMROGRAMAN MOBILE
MODUL 2

Laporan Praktikum Pemrograman Mobile Modul 2: Android Layout With Compose ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile . Laporan Prakitkum ini dikerjakan oleh:

Nama Praktikan : Hamka Arifani
NIM : 2410817110006

Menyetujui, Asisten Praktikum	Mengetahui, Dosen Penanggung Jawab Praktikum
Muhammad Erza Raihan NIM. 2310817210027	Irham Maulani Abdul Gani, S.Kom., M.Kom. NIP. 199710312025061009

DAFTAR ISI

LEMBAR PENGESAHAN i
DAFTAR ISI ii
DAFTAR GAMBAR iii
DAFTAR TABEL iv
SOAL 1 1
A. Source Code 5
B. Output Program 14
C. Pembahasan 23
SOAL 2 34
A. Pembahasan 34
Soal Essay Additional 36
A. Pembahasan Soal Additional 36
TAUTAN GIT 40
DAFTAR PUSTAKA 41

DAFTAR GAMBAR

Gambar 1. Tampilan Awal Aplikasi 2
Gambar 2. Tampilan Pilihan Persentase Tip 3
Gambar 3. Tampilan Aplikasi Setelah Dijalankan 4
Gambar 4. Screenshot Hasil Jawaban Soal 1 Compose Tampilan Awal 14
Gambar 5. Screenshot Hasil Jawaban Soal 1 Compose Dropdown 15
Gambar 6. Screenshot Hasil Jawaban Soal 1 Compose Tip Percentage 15% Sebelum dan Sesudah Round Up 16
Gambar 7. Screenshot Hasil Jawaban Soal 1 Compose Tip Percentage 18% Sebelum dan Sesudah Round Up 17
Gambar 8. Screenshot Hasil Jawaban Soal 1 Compose Tip Percentage 20% Sebelum dan Sesudah Round Up 18
Gambar 9. Screenshot Hasil Jawaban Soal 1 XML Tampilan Awal 19
Gambar 10. Screenshot Hasil Jawaban Soal 1 XML Dropdown 20
Gambar 11. Screenshot Hasil Jawaban Soal 1 XML Tip Percentage 15% Sebelum dan Sesudah Round Up 21
Gambar 12.Screenshot Hasil Jawaban Soal 1 XML Tip Percentage 18% Sebelum dan Sesudah Round Up 22
Gambar 13. Screenshot Hasil Jawaban Soal 1 XML Tip Percentage 20% Sebelum dan Sesudah Round Up 23

DAFTAR TABEL

Tabel 1. Source Code Soal 1 MainActivity.kt Compose 5
Tabel 2. Source Code Soal 1 MainActivity.kt XML 9
Tabel 3. Source Code Soal 1 Activity_Main.XML..... 11

SOAL 1

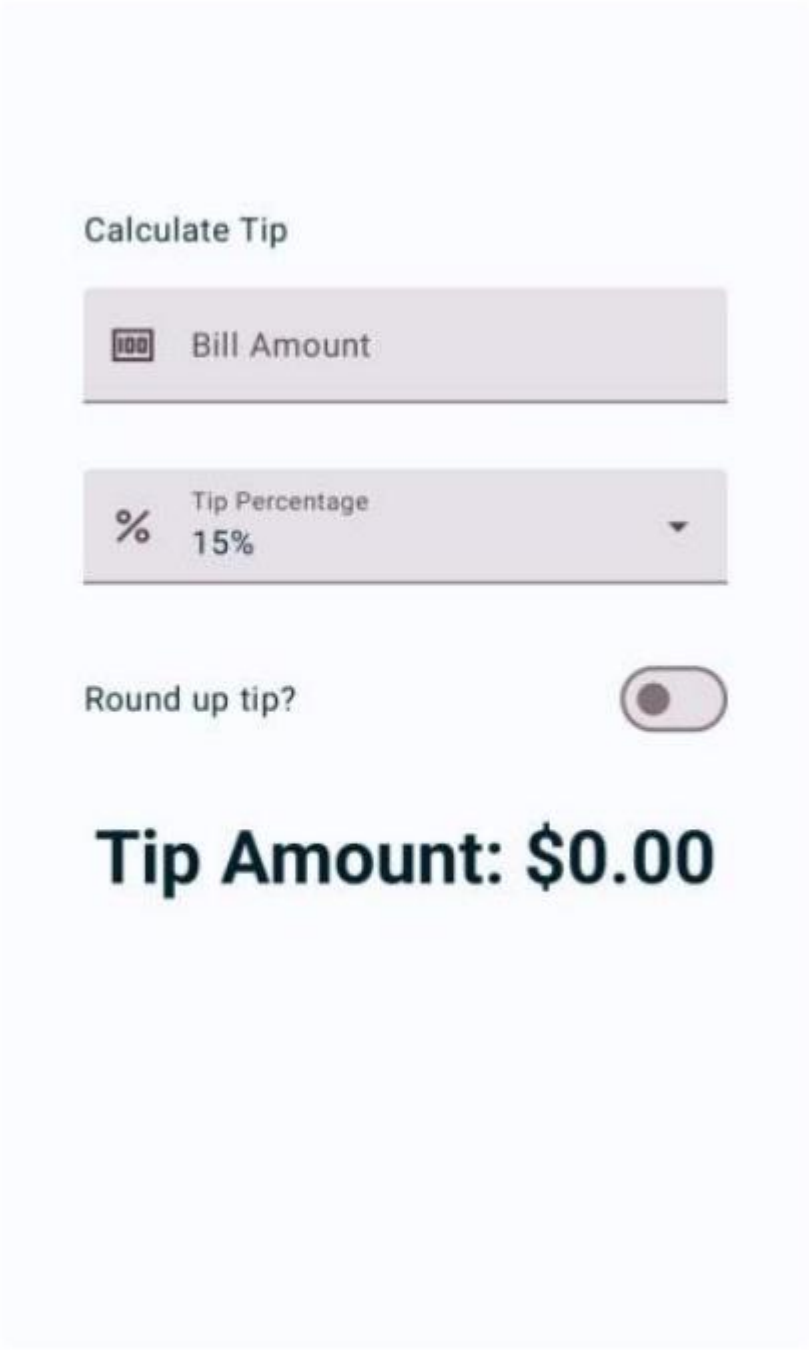
Buatlah sebuah aplikasi kalkulator tip menggunakan XML dan Jetpack Compose yang dirancang untuk membantu pengguna menghitung tip yang sesuai berdasarkan total biaya layanan yang mereka terima. Fitur-fitur yang diharapkan dalam aplikasi ini mencakup:

a. Input biaya layanan: Pengguna dapat memasukkan total biaya layanan yang diterima dalam bentuk nominal.

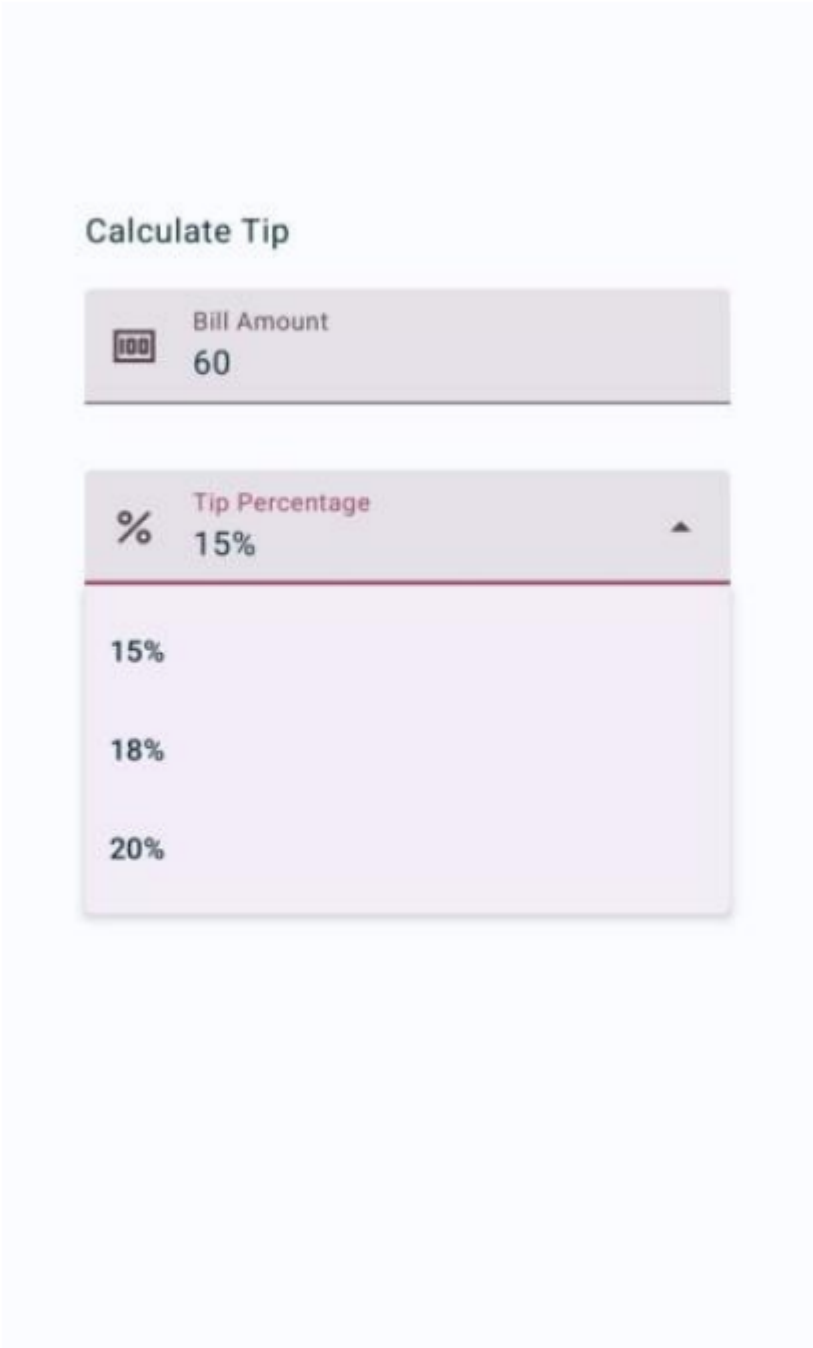
b. Pilihan persentase tip: Pengguna dapat memilih persentase tip yang diinginkan.

c. Pengaturan pembulatan tip: Pengguna dapat memilih untuk membulatkan tip ke angka yang lebih tinggi.

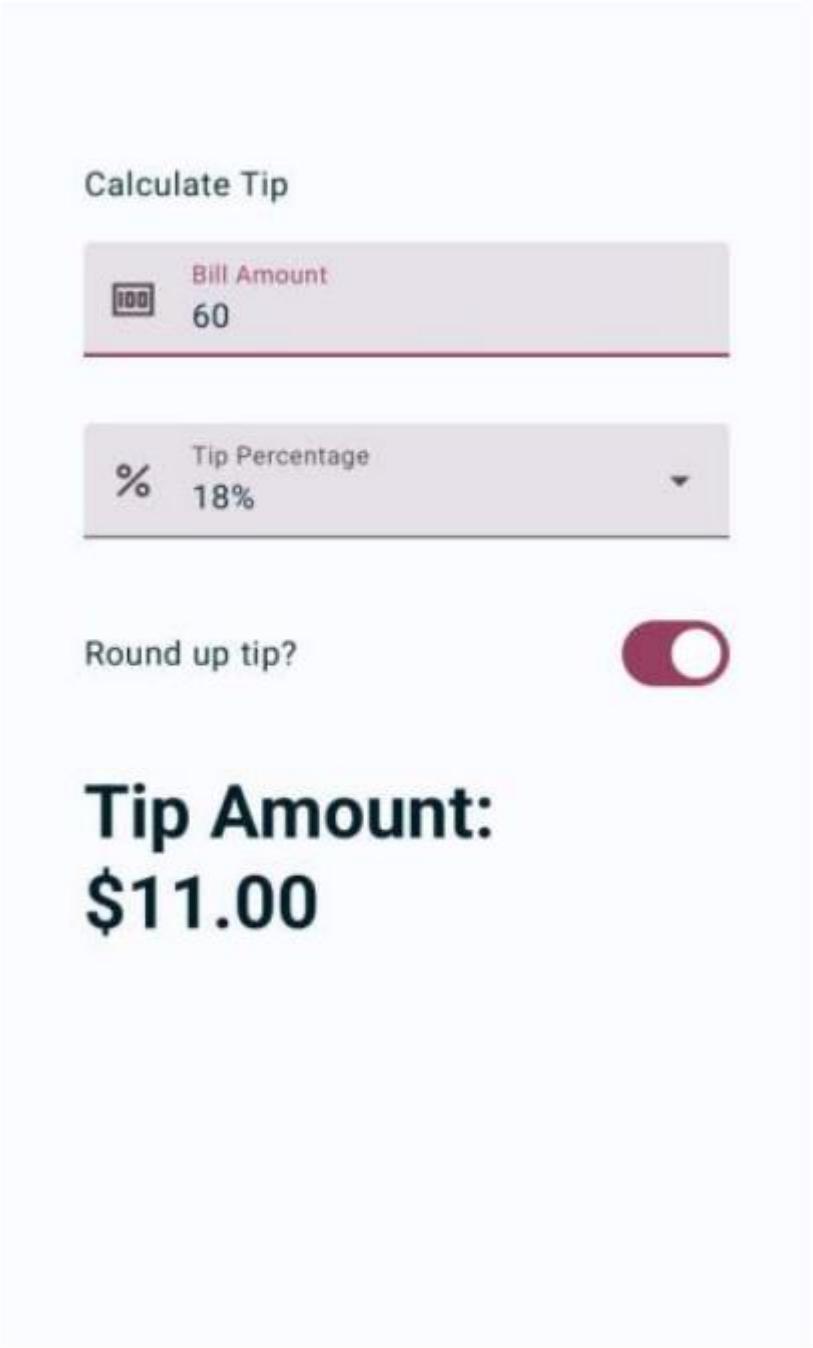
d. Tampilan hasil: Aplikasi akan menampilkan jumlah tip yang harus dibayar secara langsung setelah pengguna memberikan input.



Gambar 1. Tampilan Awal Aplikasi



Gambar 2. Tampilan Pilihan Persentase Tip



Gambar 3. Tampilan Aplikasi Setelah Dijalankan

A. Source Code

MainActivity.kt Compose

Tabel 1. Source Code Soal 1 MainActivity.kt Compose

```
package com.example.tipcalculator

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.rememberScrollState
import androidx.compose.foundation.text.KeyboardOptions
import androidx.compose.foundation.verticalScroll
import androidx.compose.material3.DropdownMenuItem
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Surface
import androidx.compose.material3.Text
import androidx.compose.material3.TextField
import androidx.compose.runtime.Composable
import androidx.compose.runtime.mutableStateOf
import androidx.compose.ui.Modifier
import androidx.compose.runtime.remember
import androidx.compose.ui.res.stringResource
import java.text.NumberFormat
import
com.example.tipcalculator.ui.theme.TipCalculatorTheme
import kotlin.math.ceil
import androidx.compose.ui.text.input.KeyboardType
import androidx.compose.ui.text.input.ImeAction
import
androidx.compose.material3.ExperimentalMaterial3Api
import androidx.compose.material3.ExposedDropdownMenuBox
import
androidx.compose.material3.ExposedDropdownMenuDefaults
import androidx.compose.material3.Icon
import androidx.compose.material3.MenuAnchorType
import androidx.compose.material3.Switch
import androidx.compose.ui.Alignment
import androidx.compose.ui.res.painterResource
import androidx.compose.ui.unit.dp

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
```

```
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            TipCalculatorTheme {
                Surface(
                    modifier = Modifier.fillMaxSize(),
                    color = MaterialTheme.colorScheme.background
                ) {
                    TipCalculatorApp()
                }
            }
        }
    }
}

@Composable
fun TipCalculatorApp(modifier: Modifier= Modifier){
    val amountInput = remember { mutableStateOf("") }
    val tipPercent = remember { mutableStateOf(15.0)}
    val roundUp = remember {mutableStateOf(false)}

    val amount = amountInput.value.toDoubleOrNull() ?: 0.0
    val tip = calculateTip(amount, tipPercent.value, roundUp.value)

    Column(
        modifier=modifier
            .padding(32.dp)
            .verticalScroll(rememberScrollState()),
        verticalArrangement = Arrangement.spacedBy(16.dp)

    ) {
        Text(text = stringResource(R.string.calculate_tip))

        BillField(
            value = amountInput.value,
            onValueChange = {newValue -> amountInput.value = newValue}
        )
        TipDropdown(
            percentage = tipPercent.value,
            onPercentageChange = {newPercentage -> tipPercent.value = newPercentage}
        )
        RoundUpSwitch(
```

```
            clicked = roundUp.value,
            onClicked = {newStatus -> roundUp.value = newStatus}
        )
        Text(
            text = stringResource(R.string.tip_amount, tip),
            style = MaterialTheme.typography.displaySmall
        )
    }
}

@Composable
fun BillField (
    modifier: Modifier = Modifier,
    value : String,
    onValueChange: (String) -> Unit
){
    TextField(
        value = value,
        leadingIcon = {
            Icon(
                painter= painterResource(R.drawable.money),
                contentDescription = null
            )
        },
        onValueChange = onValueChange,
        label = {Text(stringResource(R.string.bill_amount))},
        singleLine = true,
        keyboardOptions = KeyboardOptions(
            keyboardType = KeyboardType.Number,
            imeAction = ImeAction.Done
        ),
        modifier = modifier
    )
}

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun TipDropdown(
    modifier: Modifier = Modifier,
    percentage : Double,
    onPercentageChange : (Double)-> Unit
) {
    val expanded = remember { mutableStateOf(false) }
    val options = listOf(15.0, 18.0, 20.0)
```

```

    ExposedDropdownMenuBox(
        expanded = expanded.value,
        onExpandedChange = {newValue->expanded.value=newValue},
        modifier = modifier
    ) {
        TextField(
            readOnly = true,
            leadingIcon = {
                Icon(
                    painter=painterResource(R.drawable.percentage),
                    contentDescription = null
                )
            },
            trailingIcon = {
ExposedDropdownMenuDefaults.TrailingIcon(expanded = expanded.value)
            },
            value = "${percentage.toInt()}%",
            onValueChange= {},
            colors = ExposedDropdownMenuDefaults.textFieldColors(),
            modifier = Modifier.menuAnchor(type=MenuAnchorType.PrimaryNotEditable),
            label = {Text(stringResource(R.string.tip_percentage))}
        )
        ExposedDropdownMenu(
            expanded = expanded.value,
            onDismissRequest = {expanded.value=false}
        ) {
            options.forEach {options->
                DropdownMenuItem(
                    text = {Text("${options.toInt()}%")},
                    onClick = {
                        onPercentageChange(options)
                        expanded.value=false
                    }
                )
            }
        }
    }
}

@Composable
```

```
fun RoundUpSwitch(
    modifier: Modifier = Modifier,
    clicked : Boolean,
    onClicked : (Boolean)->Unit
){
    Row(
        modifier = modifier.fillMaxWidth(),
        horizontalArrangement = Arrangement.SpaceBetween,
        verticalAlignment = Alignment.CenterVertically
    ) {
        Text(text = stringResource(R.string.round_up_tip))
        Switch(
            checked = clicked,
            onCheckedChange = onClicked
        )
    }
}

fun calculateTip(amount : Double, tipPercent : Double, roundUp : Boolean): String{
    val tip = tipPercent / 100 * amount
    val finalTip = if (roundUp) ceil(tip) else tip
    return NumberFormat.getCurrencyInstance().format(finalTip)
}
```

MainActivity.kt XML

Tabel 2. Source Code Soal 1 MainActivity.kt XML

```
package com.example.tipxml

import android.os.Bundle
import android.widget.ArrayAdapter
import androidx.activity.enableEdgeToEdge
import androidx.appcompat.app.AppCompatActivity
import com.example.tipxml.databinding.ActivityMainBinding
import java.text.NumberFormat
import kotlin.math.ceil
import androidx.core.widget.doAfterTextChanged

class MainActivity : AppCompatActivity() {
    private val binding by lazy { ActivityMainBinding.inflate(layoutInflater) }
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
```

```
        enableEdgeToEdge()
        setContentView(binding.root)
        tipCalculatorApp(binding)
    }
}
private fun tipCalculatorApp(binding: ActivityMainBinding){
    billField(binding){
        calculateAndDisplay(binding)
    }
    tipDropDown(binding){
        calculateAndDisplay(binding)
    }
    roundUpSwitch(binding){
        calculateAndDisplay(binding)
    }
}

private fun billField(binding: ActivityMainBinding, onValueChange : (String)-> Unit){
    binding.billEditText.doAfterTextChanged { onValueChange(it.toString()) }
}

private fun tipDropDown(binding: ActivityMainBinding, onPercentageChange: (Double)->Unit){
    val options = listOf(15.0, 18.0, 20.0)
    val optionsUi = options.map { "${it.toInt()}%" }
    val adapter = ArrayAdapter(
        binding.root.context,
        android.R.layout.simple_list_item_1,
        optionsUi
    )

    binding.tipDropDownText.setAdapter(adapter)
    binding.tipDropDownText.setText(optionsUi[0], false)
    binding.tipDropDownText.setOnItemClickListener{_, _, position, _ ->
        val selectedOptions = options[position]
        onPercentageChange(selectedOptions)
    }
}
private fun roundUpSwitch(binding: ActivityMainBinding,onSwitchedChange: (Boolean)-> Unit){
    binding.roundUpSwitch.setOnCheckedChangeListener { _, isChecked ->
        onSwitchedChange(isChecked)
    }
}
```

```

private fun calculateAndDisplay(binding: ActivityMainBinding){
    val amountInput = binding.billEditText.text.toString()
    val amount = amountInput.toDoubleOrNull() ?: 0.0

    val percentageText = binding.tipDropDownText.text.toString().replace("%", "")
    val percentage = percentageText.toDoubleOrNull() ?: 15.00

    val isRounded = binding.roundUpSwitch.isChecked

    val tipResult = calculateTip(amount, percentage, isRounded)
    binding.tipResultText.text = binding.root.context.getString(R.string.tip_amount, tipResult)

}

private fun calculateTip(amount : Double, tipPercent : Double, roundUp : Boolean): String{
    val tip = tipPercent / 100 * amount
    val finalTip = if (roundUp) ceil(tip) else tip
    return NumberFormat.getCurrencyInstance().format(finalTip)
}
```

Activity_Main.XML

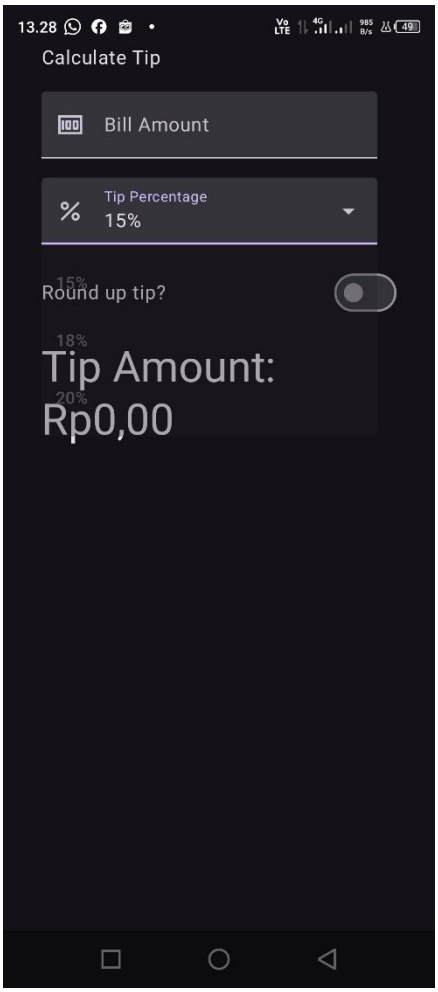
Tabel 3. Source Code Soal 1 Activity_Main.XML

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:padding="32dp"
    tools:context=".MainActivity">

    <TextView
        android:id="@+id/titleText"
```

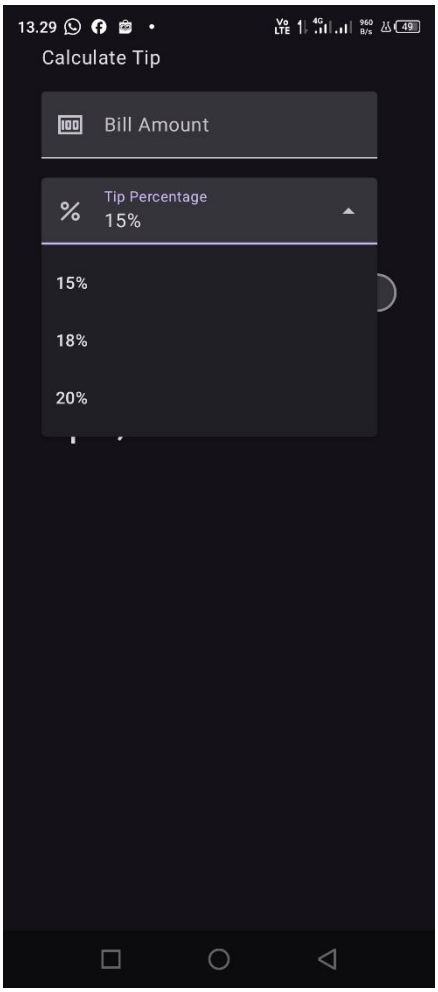
```
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/calculate_tip"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />

  <com.google.android.material.textfield.TextInputLayout
        android:id="@+id/billInputLayout"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_marginTop="16dp"
        android:hint="@string/bill_amount"
        app:startIconDrawable="@drawable/money"
     app:layout_constraintTop_toBottomOf="@id/titleText"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent">
<com.google.android.material.textfield.TextInputEditText
            android:id="@+id/billEditText"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:inputType="numberDecimal">
</com.google.android.material.textfield.TextInputEditTex
t>
</com.google.android.material.textfield.TextInputLayout>

<com.google.android.material.textfield.TextInputLayout
        android:id="@+id/tipDropDownLayout"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_marginTop="16dp"
style="@style/Widget.Material3.TextInputLayout.OutlinedB
ox.ExposedDropdownMenu"
        android:hint="@string/tip_percentage"
        app:startIconDrawable="@drawable/percent"
app:layout_constraintTop_toBottomOf="@id/billInputLayout
"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent">
        <AutoCompleteTextView
            android:id="@+id/tipDropDownText"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:inputType="none">
        </AutoCompleteTextView>
</com.google.android.material.textfield.TextInputLayout>

<com.google.android.material.switchmaterial.SwitchMateri
al
        android:id="@+id/roundUpSwitch"
```

```
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
app:layout_constraintTop_toBottomOf="@id/tipDropDownLayout"
        app:layout_constraintEnd_toEndOf="parent">
</com.google.android.material.switchmaterial.SwitchMaterial>

    <TextView
        android:id="@+id/roundUpText"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:text="@string/round_up_tip"
        android:textStyle="bold"
        android:textSize="15sp"
        app:layout_constraintStart_toStartOf="parent"
app:layout_constraintEnd_toStartOf="@id/roundUpSwitch"
    app:layout_constraintTop_toTopOf="@id/roundUpSwitch"
app:layout_constraintBottom_toBottomOf="@id/roundUpSwitch"
        />

    <TextView
        android:id="@+id/tipResultText"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="16dp"
        android:textStyle="bold"
        android:textSize="30sp"
        app:layout_constraintStart_toStartOf="parent"
app:layout_constraintTop_toBottomOf="@id/roundUpSwitch"/>
</androidx.constraintlayout.widget.ConstraintLayout>
```

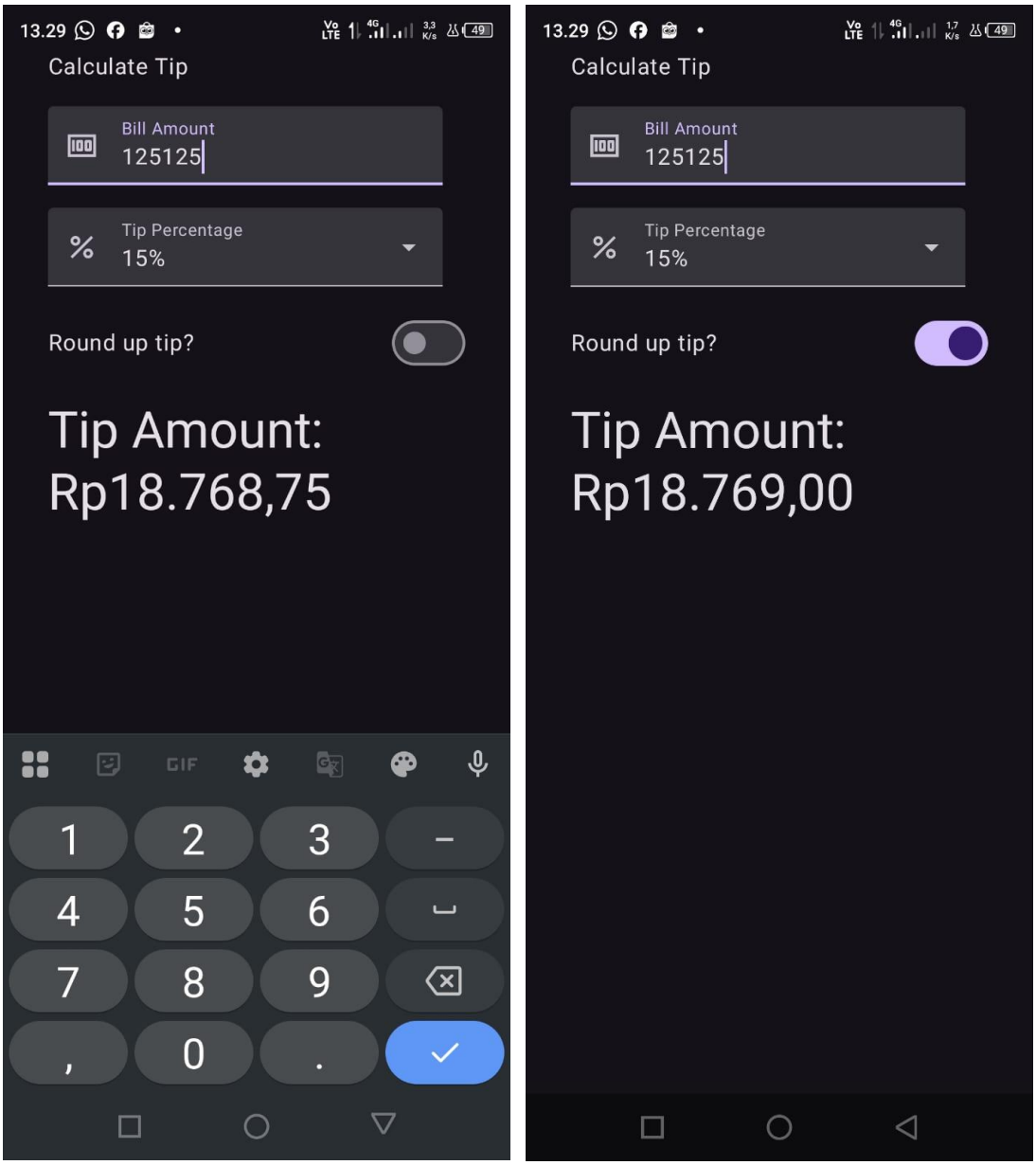
B. Output Program



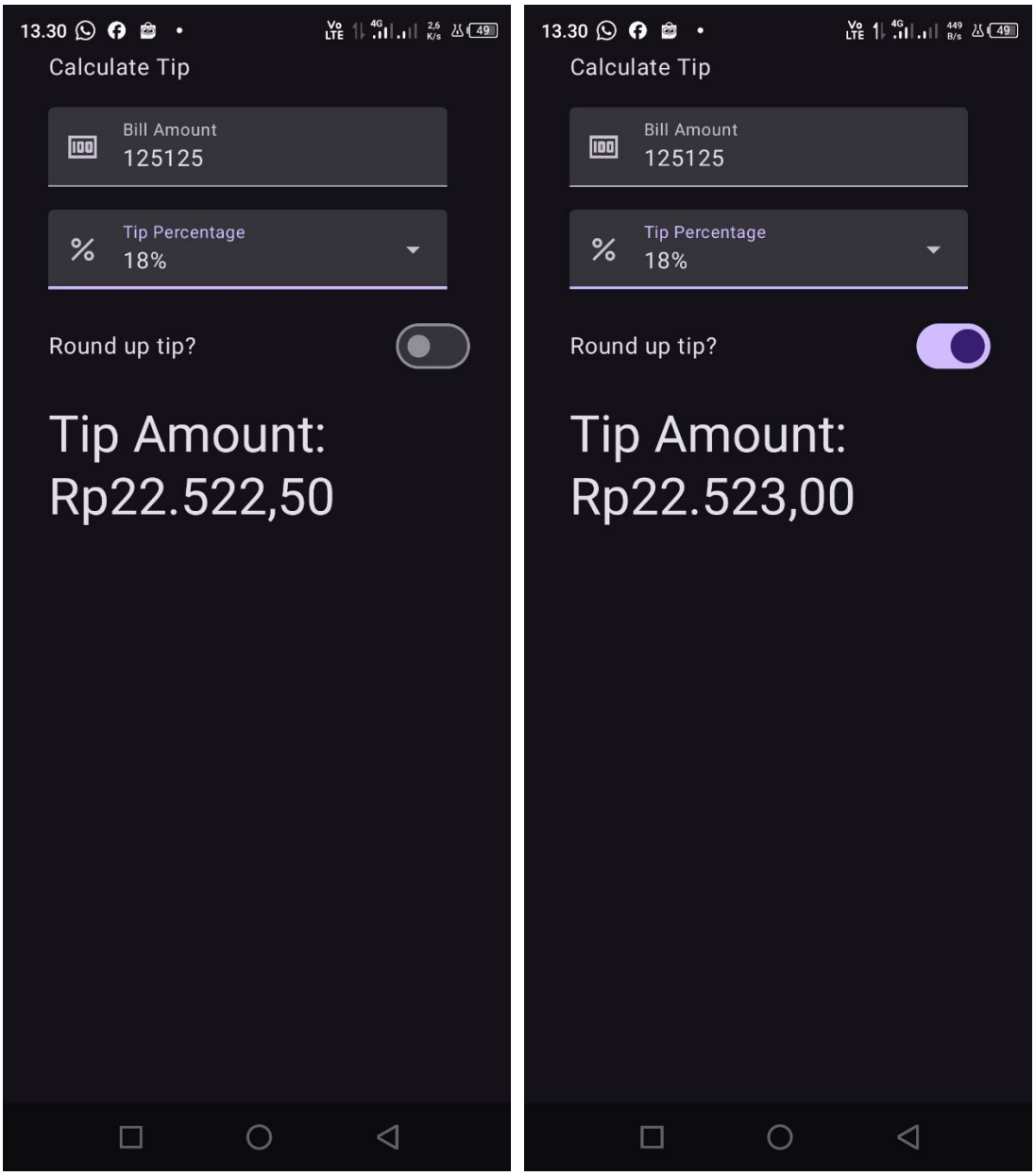
Gambar 4. Screenshot Hasil Jawaban Soal 1 Compose Tampilan Awal



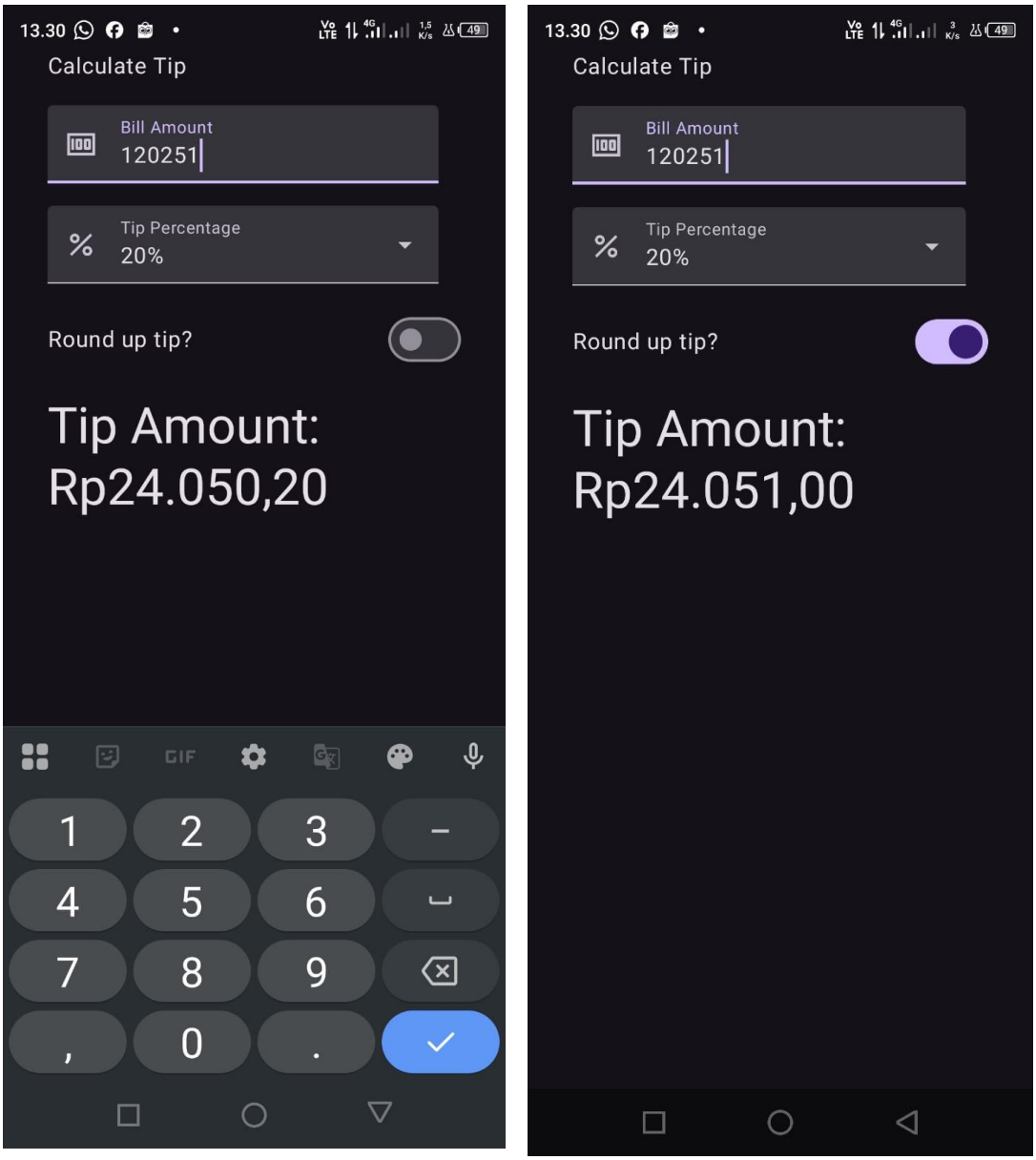
Gambar 5. Screenshot Hasil Jawaban Soal 1 Compose Dropdown



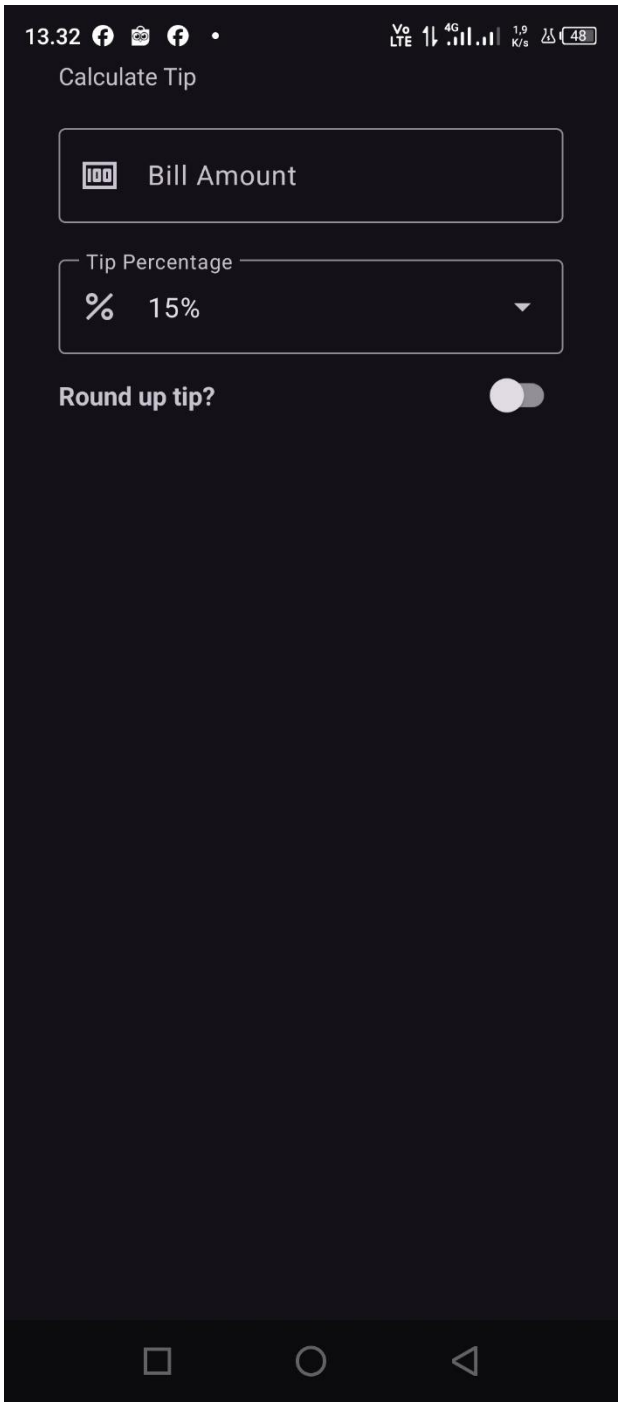
Gambar 6. Screenshot Hasil Jawaban Soal 1 Compose Tip Percentage 15% Sebelum dan Sesudah Round Up



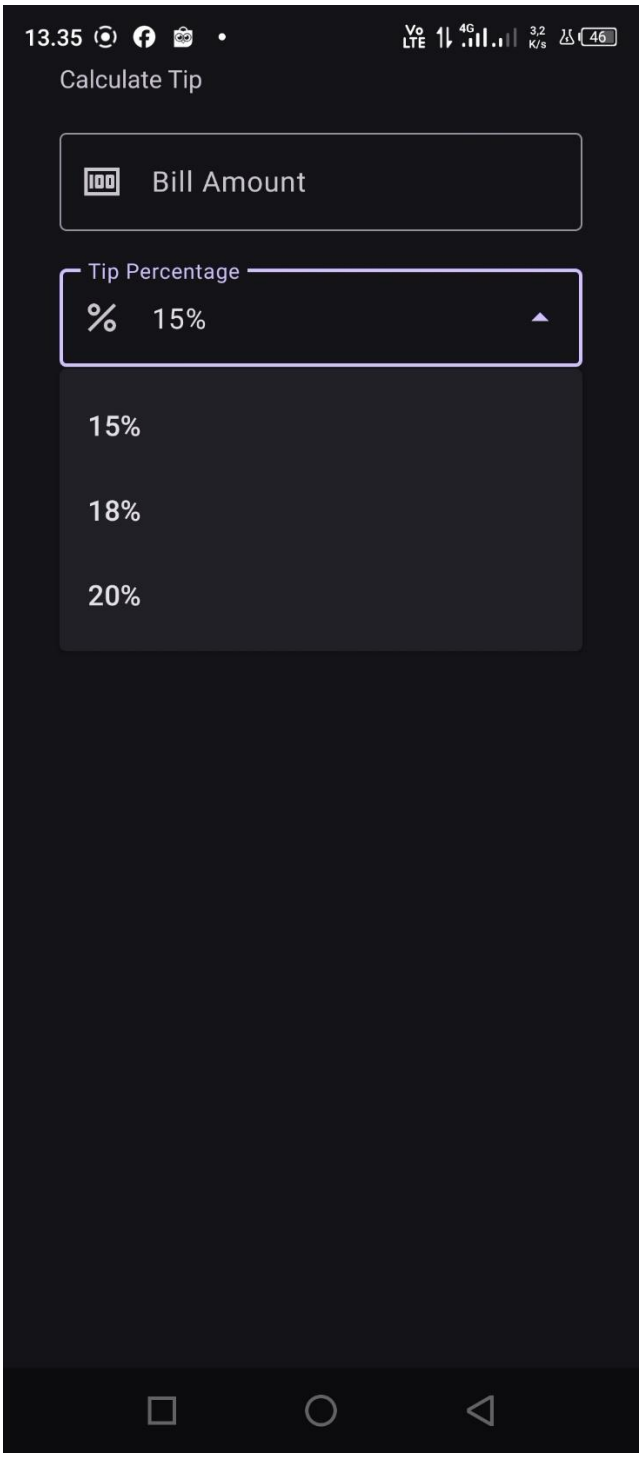
Gambar 7. Screenshot Hasil Jawaban Soal 1 Compose Tip Percentage 18% Sebelum dan Sesudah Round Up



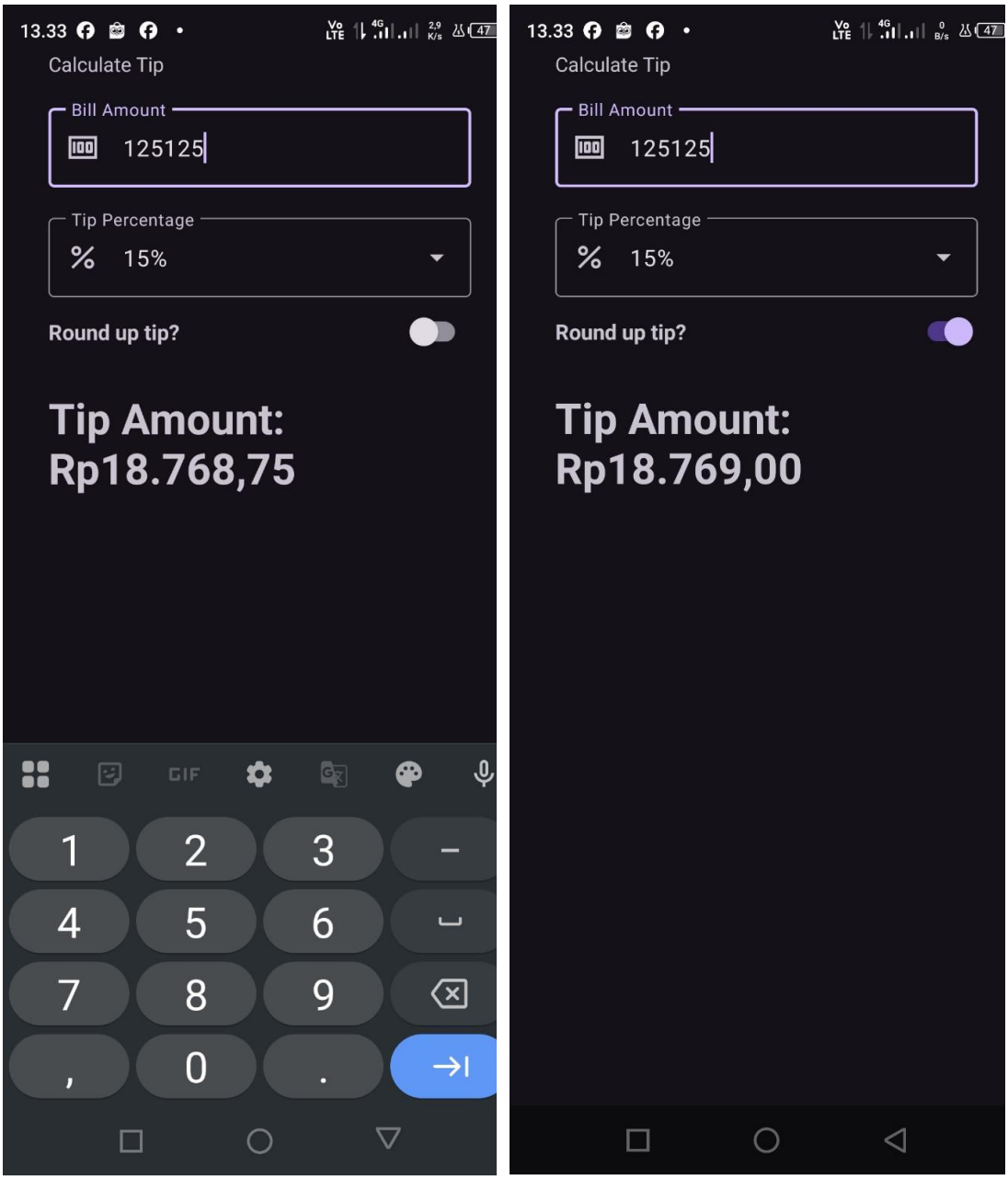
Gambar 8. Screenshot Hasil Jawaban Soal 1 Compose Tip Percentage 20% Sebelum dan Sesudah Round Up



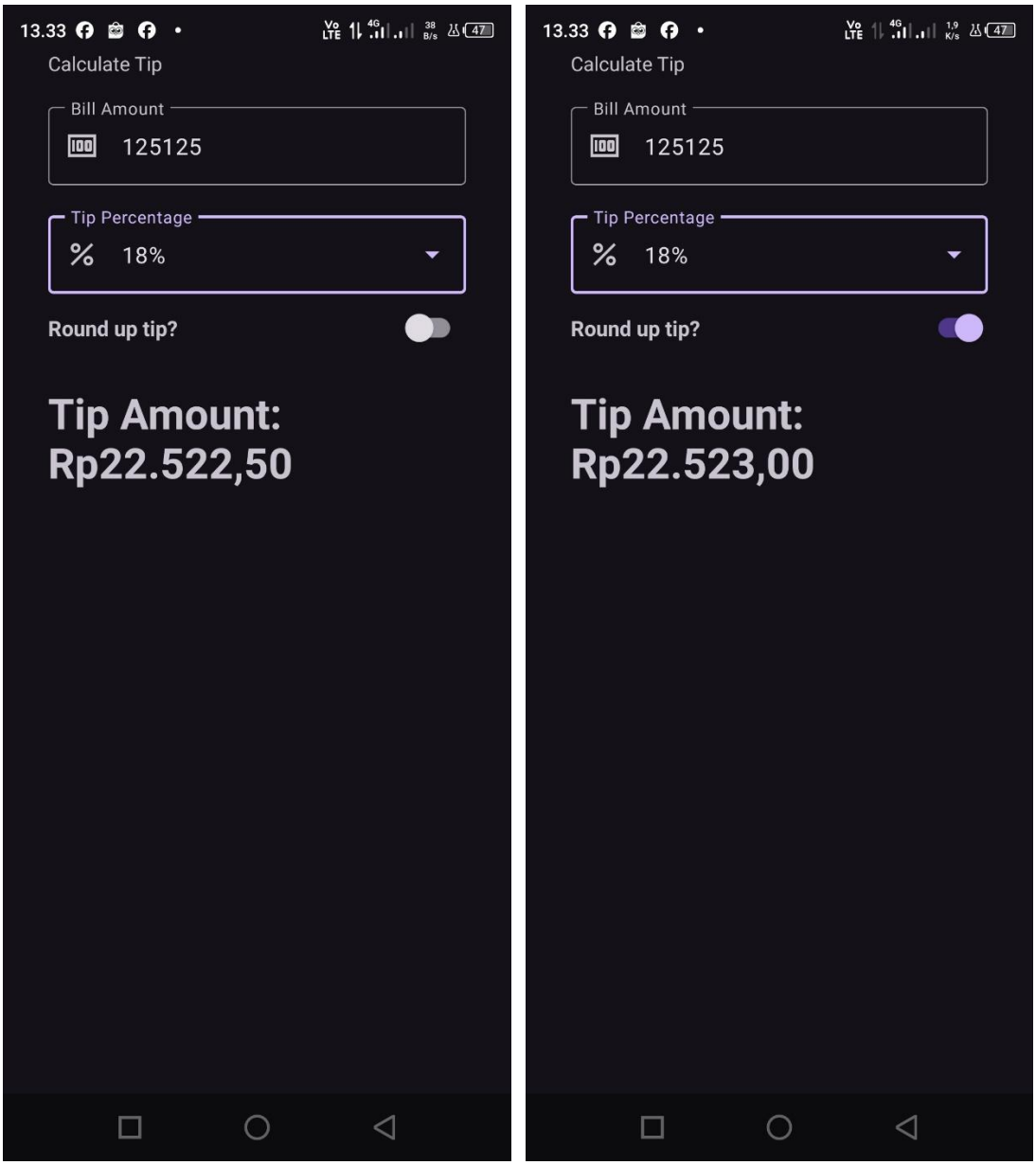
Gambar 9. Screenshot Hasil Jawaban Soal 1 XML Tampilan Awal



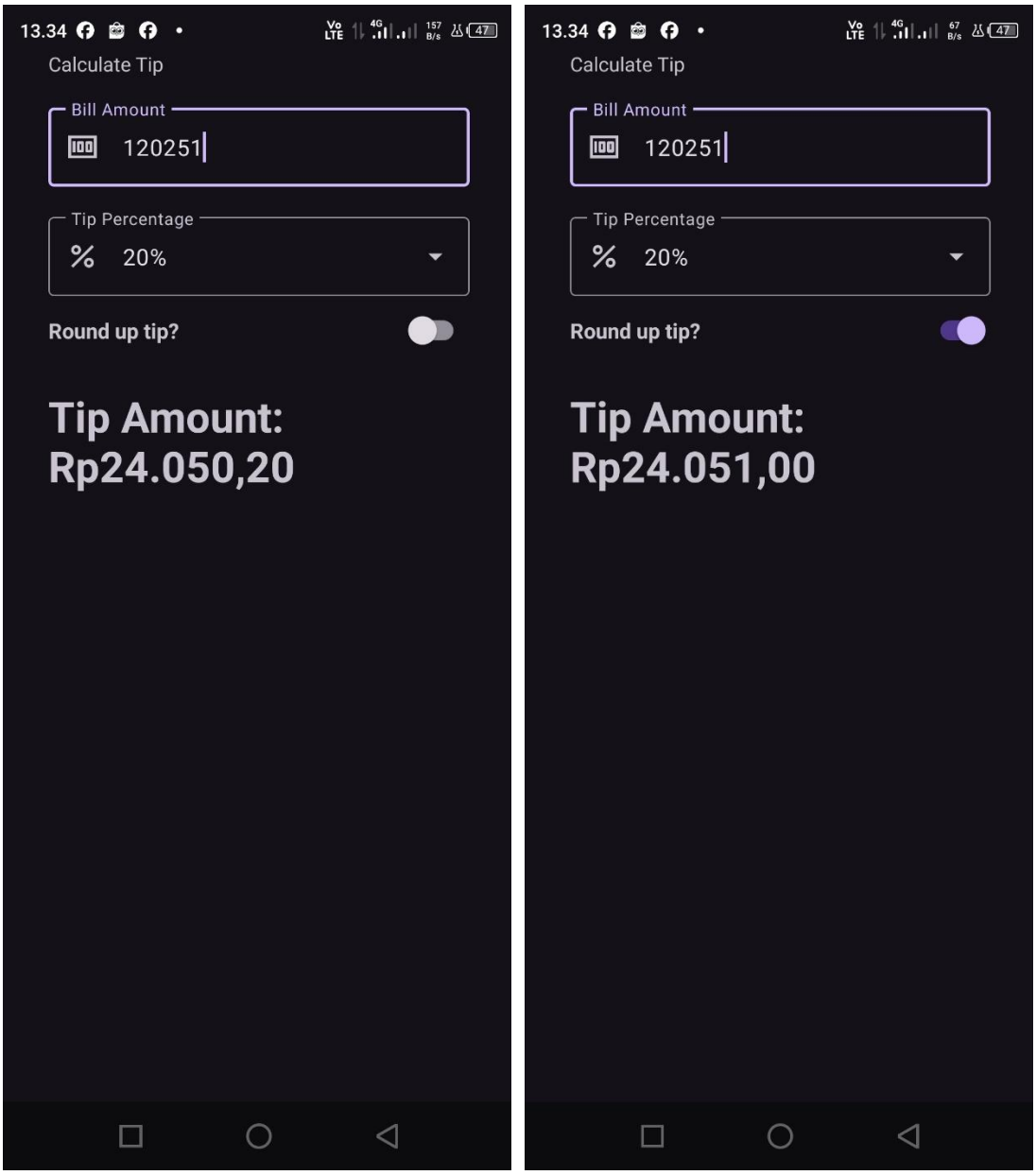
Gambar 10. Screenshot Hasil Jawaban Soal 1 XML Dropdown



Gambar 11. Screenshot Hasil Jawaban Soal 1 XML Tip Percentage 15% Sebelum dan Sesudah Round Up



Gambar 12.Screenshot Hasil Jawaban Soal 1 XML Tip Percentage 18% Sebelum dan Sesudah Round Up



Gambar 13. Screenshot Hasil Jawaban Soal 1 XML Tip Percentage 20% Sebelum dan Sesudah Round Up

C. Pembahasan

MainActivity.kt Compose

Pada baris 1, terdapat ‘package.com.example.tipcalculator’ yang berfungsi untuk memudahkan organisir projek dengan mengelompokkan file yang saling berhubungan satu sama lain ke dalam 1 folder package.

Pada baris 3 hingga 39 terdapat seluruh import dari projek ini untuk memanggil library, package, ataupun material yang diperlukan pada program, seperti salah satunya adalah ‘ceil()’ yang diimport dari package ‘kotlin.math’.

Pada baris 41 hingga 56 terdapat class ‘MainActivity’ yang secara dasar berfungsi sebagai tempat awal dimulainya progam. Cara kerja dari class ‘MainActivity’ adalah melakukukan inheritance kepada class ‘ComponentActivity()’ dan melakukan override pada function ‘oncreate(savedInstanceState)’, di mana function diisi dengan function ‘onCreate(savedInstanceState)’ dari parent yang berfungsi untuk memeriksa aktivitas sebelumnya yang telah dilakukan user guna menyiapkan tampilan UI ketika aplikasi akan dijalankan. Kemudian terdapat enableEdgetoEdge() yang bertugas memastikan tampilan aplikasi akan memenuhi layar yang tersedia, lalu ada ‘setContent’ yang berfungsi sebagai tempat untuk mendeklarasikan tampilan UI, di mana di dalamnya terdapat Theme dan juga ‘Surface’ yang bekerja berdampingan dalam mengatur tampilan UI seperti warna misalnya, di mana Theme yang akan menentukan warna contohnya jika pada projek ini warna akan disesuaikan dengan tema latar dari handphone pengguna apakah dalam keadaan dark mode atau light mode, sementara ‘Surface’ lah yang akan mengimplementasikan warnanya pada layar. Adapun pada projek ini selain mengatur warna, ‘Surface’ juga bertugas memastikan aplikasi akan memenuhi seluruh luas layar yang tersedia, dan di dalamnya dipanggil function ‘TipCalculatorApp’.

Pada baris 58 hingga 93 diinisialisasi function bernama ‘TipCalculatorApp’ yang diberi ‘@composable’ untuk menandakan jika function ini berfungsi sebagai salah satu dari komponen UI, function ini juga diberikan parameter ‘modifier’ sesuai standar dari jetpack compose yang berfungsi agar function menerima pengaturan tampilan dari luar, namun jika tidak ada maka akan digunakan paremeter baru bernama ‘Modifier’ yang belum memiliki isi. Pertama-tama diinisialisasi 3 variabel bernama ‘amountInput’, ‘tipPercent’, dan ‘roundUp’ bertipe ‘val’ yang masing-masingnya diberi nilai ‘remember { mutablestateof() }’ dengan tujuan untuk memberikan state atau nilai awal dari ketiga variabel. Kemudian diinisialisasi lagi dua variabel yakni ‘amount’ dan ‘tip’ yang juga bertipe val, adapun ‘amount’ diberi nilai dari variabel ‘amountInput’

menggunakan ‘.value’, sementara ‘tip’ diberi nilai hasil perhitungan tip menggunakan function calculateTip yang diberi parameter variabel ‘amount’, ‘tipPercentage’, dan ‘roundUp’.

Setelahnya pada baris 67 digunakan ‘Column()’ untuk membungkus komponen di dalamnya agar tersusun secara vertikal pada tampilan UI, ‘Column()’ diberikan parameter yakni modifier dengan pengaturan tampilan tambahan yaitu ‘padding’ sebesar 32dp untuk semua arah, dan ‘verticalScroll(rememberScrollState())’ yang berguna agar pada bagian isi dari ‘Column()’ bisa discroll oleh pengguna secara vertikal, berjaga-jaga jika saja misalnya saat pengguna menggunakan aplikasi dalam keadaan horizontal dan tampilan aplikasi menjadi terpotong sehingga harus discroll. Adapun isi dari ‘Column’ yang pertama dan akan ditampilkan di posisi paling atas adalah ‘Text()’ yang isi teksnya diatur agar menggunakan teks dari resource String bernama ‘calculate_tip’.

Kemudian dilakukan pemanggilan ketiga function yang akan diinisialisasi selanjutnya yaitu billField(), tipDropdown(), dan roundUpSwitch() yang masing-masing diberi parameter berupa variabel yang akan menyimpan nilai dari amountInput, tipPercentage, serta roundUp. Variabel tersebut harus diinisialisasi bersama dengan ‘on...Change’ yang berfungsi untuk membantu memberikan nilai baru kepada variabel pasangannya dengan menggunakan lambda expression, contohnya untuk billField diberi parameter variabel ‘value’ dengan isi lambda expression di variabel ‘onValueChange’nya adalah ‘{newValue-> amountInput.value = newValue }’ , atau yang berarti dibuat variabel baru untuk menampung input dari pengguna, lalu nilainya kemudian akan dimasukkan ke variabel amountInput, sehingga nilai dari variabel value juga akan diperbarui dan nilai pada tampilan UI juga akan berubah.

Terakhir kembali terdapat ‘Text’ yang isi teksnya diatur menggunakan ‘stringResource(R.string.tip_amount, tip)’ atau berarti menggunakan resource string Bernama tip_amount yang diberi nilai dari variabel tip untuk menggantikan symbol $ pada isi dari tip_amount, kemudian diberikan pengaturan ‘style=Materialtheme.

typhography.displaysmall' sehingga font dari teks akan menjadi Roboto dengan ukuran font 36.

Pada baris 95 hingga 118 terdapat inisialisasi function bernama 'BillField()' yang juga diberi @composable dan parameter modifier seperti function 'TipCalculatorApp()', namun diberi tambahan parameter yakni variabel 'value' bertipe 'Double' dan pasangan variabelnya untuk memberi nilai baru pada 'value' jika terjadi perubahan yakni variabel 'onValueChange' yang diberi nilai '(String) -> Unit' atau berarti variabel akan menerima nilai bertipe Double dan tidak akan mengembalikan nilai pada saat dipanggil karena tugasnya hanya memberi nilai baru pada 'value'. Pada 'BillField()' digunakan 'TextField()' untuk membuat kolom teks yang bisa menerima input dari pengguna, dengan di dalamnya diatur agar nilai yang ditampilkan pada kolom teks nantinya adalah nilai dari 'value', kemudian diberikan icon menggunakan resource drawable bernama 'money' menggunakan 'painterResource', lalu 'onValueChange' yang mengatur apa yang akan dilakukan ketika nilainya berubah diisi dengan variabel 'onValueChange' yang menjadi parameter sebelumnya, diberi label berupa teks menggunakan resource string bernama 'bill_amount' menggunakan 'stringResource', diatur agar hanya menjadi satu baris menggunakan 'singleLine=true', diberi pengaturan tampilan menggunakan modifier, dan diberi 'keyboardOptions' untuk mengatur tipe keyboard yang muncul pada handphone pengguna ketika akan memasukkan input menjadi keyboard bertipe number, dan tombol di pojok kanan bawah menjadi done.

Pada baris 120 hingga 167, terdapat inisialisasi function bernama 'TipDropdown' yang diberi @composable dan parameter function seperti dua function sebelumnya, namun karena penggunaan 'exposedDropdownMenu()' pada isi dari function ini, di mana komponen tersebut masih dianggap belum sepenuhnya selesai dan masih bisa terjadi perubahan di masa depan. Function ini juga diberi parameter variabel bernama 'percentage' bertipe 'Double' dan pasangan variabelnya bernama 'onPercentageChange' yang bertugas memberi nilai baru kepada variabel 'percentage', karenanya variabel tersebut diatur agar menerima input bertipe 'Double', dan diatur

tidak mengembalikan nilai dengan '-> Unit'. Pertama-tama pada function ini diinisialisasi variabel bernama 'expanded' yang diatur nilainya menggunakan 'remember { mutableStateOf(false) }' untuk memberi nilai awal yakni false dan akan diingat nilainya oleh 'expanded', serta ''options' yang diberi nilai menggunakan 'listOf()' untuk menampung tiga nilai opsi yakni 15, 18, dan 20 dengan tipe 'Double'. Kemudian digunakan 'exposedDropdownMenu()' dari material3 untuk membuat menu yang bisa terbuka ke bawah seperti pada soal, di mana 'exposedDropdownMenu' diberikan parameter 'expanded' untuk menerima nilai dari variabel 'expanded' menggunakan '.value', dan 'onExpandedChange' yang bertugas menampung nilai ketika ada input dari pengguna saat salah satu opsi dropdown dipilih lalu akan diteruskan ke variabel 'expanded', selain itu 'exposedDropdownMenuBox()' juga diberi pengaturan tampilan menggunakan 'modifier'.

Adapun isi dari 'exposedDropdownMenuBox' adalah 'TextField()' untuk membuat kolom teks yang diatur agar pengguna hanya bisa memilih salah satu dari opsi pada dropdown dan bukan mengetikkan input secara manual. Lalu diberikan icon menggunakan resource drawable bernama 'percentage', dan diberikan ikon di bagian ujung kanan kolom teksnya menggunakan 'trailingIcon' di mana 'TrailingIcon' dari 'exposedDropdownMenuBox' diatur agar menerima nilai dari variabel 'expanded' sehingga ikon bisa berputar secara otomatis dengan mendeteksi apakah pengguna membuka dropdown atau tidak. Selanjutnya 'TextField' diberi 'value' yakni nilai dari variabel 'percentage' yang dikonversi ke 'Int' menggunakan '.toInt()', sementara 'onValueChange'-nya dikosongkan karena pengguna memang tidak bisa memberikan input baru. Kemudian 'TextField' diberikan pengaturan warna agar menyesuaikan secara otomatis dengan pengaturan warna pada 'Theme', lalu diberi 'modifier' berupa 'menuAnchor' agar menu dropdown dan juga kolom teks menjadi satu kesatuan, serta tipenya diatur ke 'primaryNotEditable' yang semakin mempertegas jika menu dropdown ini tidak bisa dirubah, tipe pada 'menuAnchor' sendiri baru bisa dikatakan baru ditambahkan karena penggunaan 'menuAnchor' biasa sudah defricated. Terakhir diberikan label menggunakan resource String bernama tip_percentage.

Setelahnya pada isi dari ‘exposedDropdownMenu’, terdapat ‘exposedDropdownMenu’ yang berfungsi untuk mengatur isi dari dropdown, di mana diberikan parameter yakni ‘expanded’ yang menampung nilai dari variabel ‘expanded’ menggunakan ‘.value’, lalu ‘onDismissRequest’ yang merupakan parameter bawaan dari ‘exposedDropdownMenu’ untuk mengatur apa yang akan terjadi ketika pengguna memilih untuk menutup menu dropdown yakni nilai dari variabel ‘expanded’ menjadi false. Adapun isi dari variabel ini adalah pengulangan ‘forEach’ yang berfungsi agar untuk setiap nilai dari variabel ‘options’ akan dibuat menjadi ‘exposedDropdownItems’ atau opsi dari dropdown, yang mana setiap itemnya akan diberikan teks berupa konversi nilai dari variabel ‘options’ tersebut ke bentuk Int, dan diatur agar ketika pengguna melakukan klik maka ‘onPercentageChange’ diberi nilai variabel ‘options’ yang dipilih oleh pengguna dan akan diberikan kepada variabel ‘percentage’ sehingga tampilan akan diperbarui serta menu dropdown diatur agar tertutup.

Pada baris 169 hingga 186 terdapat inisialisasi function bernama ‘roundUpSwitch’ yang sama seperti function lainnya juga diberi ‘@composable’, ‘modifier’, serta 2 variabel sebagai parameter yakni ‘clicked’ dan ‘onClicked’ yang menerima input bertipe ‘Boolean’. Adapun isi dari function ini adalah ‘Row’ yang berfungsi untuk menyusun komponen di dalamnya secara horizontal, ‘Row’ diberi parameter berupa ‘modifier.fillMaxWidth()’, yang mengatur agar isi komponennya akan memanfaatkan sepenuhnya luas layer yang tersedia, serta mengatur posisi komponennya agar secara vertical berada di tengah, namun secara horizontal memiliki jarak antar komponennya. Adapum isi komponen dari ‘Row’ adalah ‘Text’ yang isi teksnya diatur untuk menampilkan isi dari resource string bernama ‘round_up_tip’, dan ‘Switch’ untuk membuat tombol di mana yang bisa dilakukan switch, di mana tombolnya diatur agar saat ditekan maka nilai variabel ‘checked’ bawaanya akan diberi nilai dari variabel ‘clicked’, begitu pula dengan variabel bawaan lainnya yakni ‘onCheckedChange’ yang diatur untuk mengirimkan nilainya pada ‘onClicked’ sehingga nilai dari ‘clicked’ juga akan diperbarui beserta dengan tampilan UI.

Pada baris 188 hingga 192 terdapat function bernama ‘calculateTip’ yang diberi parameter yakmi variabel ‘amount’ dengan tipe ‘Double’, ‘tipPercent’ dengan tipe ‘Double’, dan variabel ‘roundUp’ bertipe ‘Boolean’, dan diatur agar nantinya akan mengembalikan nilai bertipe ‘String’ dengan ‘: String’. Pertama-tama pada function diinisialisasi variabel ‘tip’ yang diberi nilai hasil dari ‘tipPercent/100 * amount’, kemudian diinisialisasi variabel finalTip yang nilainya diatur dengan penjikaan yakni apabila roundUp bernilai true maka nilai dari finalTip adalah hasil pembulatan dari variabel ‘tip’ menggunakan ‘ceil()’ yang berasal dari package Kotlin.math, sementara jika nilainya false maka nilai dari finalTip akan mengambil nilai dari variabel tip sesuai perhitungan sebelumnya. Barulah dilakukan pemformatan kepada nilai dari variabel ‘finalTip’ ke nominal mata uang lokal yang pada kasus ini adalah Rupiah Indonesia, dan hasilnya akan menjadi nilai yang dikembalikan oleh function ‘calculateTip’.

MainActivity.kt XML

Pada baris 1, terdapat ‘package com.example.tipxml’ yang berfungsi untuk memudahkan organisir projek dengan mengelompokkan file yang saling berhubungan satu sama lain ke dalam 1 folder package.

Pada baris 3 hingga hingga 10, terdapat semua import yang digunakan pada projek untuk memanggil komponen, library, atau package yang dibutuhkan dalam proses pembuatan aplikasi tip calculator, seperti salah satunya sama seperti versi compose yakni ‘ceil()’ yang diimport dari package ‘kotlin.math’yang dipanggil agar pada program bisa membuat tombol.

Sama seperti versi compose, pada versi XML juga terdapat MainActivity yang memiliki tugas dasar yang sama persis namun terdapat perbedaan dalam implementasinya. Salah satu perbedaannya adalah MainActivity versi XML melakukan inheritance dari AppCompatActivity() yang berasal dari library, kemudian karna pada projek ini diatur agar bisa menggunakan viewBinding, maka perlu dibuat sebuah variabel bernama binding bertipe val yang bertugas untuk menjalankan ‘ActivityMainBinding.inflate(layoutInflater)’ atau proses untuk menerjemahkan teks

XML ke bentuk tampilan UI. Penggunaan ‘by Lazy’ berguna agar program tidak langsung menjalankan ‘layoutInflater’, dikarenakan sistem activity belum siap sepenuhnya karena inisialisasi dilakukan sebelum function onCreate(), sehingga ketika objek ‘binding’ dibuat pada ‘setContentView = binding.root’ barulah proses penerjemahan dari seluruh view yang ada pada projek dijalankan. Barulah setelahnya akan pada main dipanggil function ‘TipCalculatorApp()’

Pada baris 21 hingga 31 terdapat inisialisasi function bernama ‘TipCalculatorApp’ yang diberi parameter ‘binding’ yang menampung seluruh informasi komponen view pada activity_main. Function ini bertugas untuk memanggil semua function lainnya yang diberi parameter ‘binding’ dan setiap function diatur untuk menjalankan function ‘calculateAndDisplay’ yang juga diberi parameter ‘binding’ untuk mendapatkan akses kepada seluruh view pada projek.

Pada baris 33 hingga 35 diinisialisasi function bernama ‘billField’ yang diberi parameter ‘binding’ dan ‘onValueChange’ yang bertugas meneruskan nilai input tanpa mengembalikan nilai apapun. Adapun function akan mendeteksi jika terjadi perubahan isi teks pada bagian kolom bill maka akan dilakukan konversi isi kolom yang baru ke bentuk String menggunakan ‘it.toString()’, untuk nantinya akan digunakan dalam proses perhitungan tip.

Pada baris 37 hingga 52 diinisialisasi function ‘tipDropDown’ yang diberikan parameter yang secara garis besarnya sama dengan function sebelumnya namun hanya berbeda nama variabelnya, pada function ini diinisialisasi variabel untuk menyimpan nilai bertipe Double dalam bentuk list, kemudian dibuat variabel untuk menampung nilainya yang dikonversi ke Int dan ditambahkan persen untuk ditampilkan pada UI, selanjutnya dibuat adapter yang bertugas membuat kolom teks untuk setiap opsi ketika dropdown diklik, lalu kolom teks diberi opsi awal yakni 15% yang berada di indeks 0, dan ketika dilakukan klik oleh pengguna untuk memilih input, sistem diatur agar menerima informasi posisi indeksnya dan mengabaikan informasi lain, sehingga function memperoleh nilai input dalam bentuk Double yang akan diteruskan untuk menjadi pemicu dari proses perhitungan.

Pada baris 53 hingga 57 sama seperti sebelumnya function ‘roundUpSwitch’ diberikan parameter ‘binding’ dan variabel untuk meneruskan nilai, di mana isi dari function ini adalah untuk mendeteksi apakah pengguna melakukan klik kepadda tombol spinner, dan jika iya maka akan mengembalikan informasi berupa variabel yang bernilai true sambil mengabaikan satu parameter atau informasi lainnya, sehingga akan memicu proses perhitungan.

Pada baris 59 hingga 71, diinisialisasi function yang akan melakukan perhitungan dan menampilkannya ke tampilan UI bernama ‘calculateAndDisplay’, di mana function awalnya akan melakukan konversi nilai dari kolom teks bill ke bentuk ‘Double’, kemudian merubah nilai yang ditampilkan pada dropdown ke bentuk Double setelah menghapus simbol % agar dapat dilakukan perhitungan, dan menerima nilai variabel ‘isChecked’ dari roundUpSwittch untuk mengetahui apakah tombol pembulatan diklik, dan dibuatlah variabel untuk menampung nilai perhitungan tip menggunakan function ‘calculateTip’, kemudian memperbarui teks pada bagian view yang menunjukkan jumlah tip setelah dilakukan perhitungan.

Pada baris 73 hingga 77, terdapat function bernama ‘calculateTip’ yang diberi parameter yakmi variabel ‘amount’ dengan tipe ‘Double’, ‘tipPercent’ dengan tipe ‘Double’, dan variabel ‘roundUp’ bertipe ‘Boolean’, dan diatur agar nantinya akan mengembalikan nilai bertipe ‘String’ dengan ‘: String’. Pertama-tama pada function diinisialisasi variabel ‘tip’ yang diberi nilai hasil dari ‘tipPercent/100 * amount’, kemudian diinisialisasi variabel finalTip yang nilainya diatur dengan penjikaan yakni apabila roundUp bernilai true maka nilai dari finalTip adalah hasil pembulatan dari variabel ‘tip’ menggunakan ‘ceil()’ yang berasal dari package Kotlin.math, sementara jika nilainya false maka nilai dari finalTip akan mengambil nilai dari variabel tip sesuai perhitungan sebelumnya. Barulah dilakukan pemformatan kepada nilai dari variabel ‘finalTip’ ke nominal mata uang lokal yang pada kasus ini adalah Rupiah Indonesia, dan hasilnya akan menjadi nilai yang dikembalikan oleh function ‘calculateTip’.

Activity_main.xml

Pada baris 2 terdapat ‘<androidx.constraintlayout.widget.ConstraintLayout’ yang berfungsi sebagai kontainer utama untuk mengatur tata letak semua komponen di dalamnya. Adapun pada baris 3-4 terdapat baris xml yang mengatur agar program bisa mengakses library luar apabila diperlukan, dan juga memberitahu komputer bahwa file ini berisikan tampilan dari aplikasi android. Kemudian terdapat baris xml yang pertama adalah android:id untuk memberi identitas pada layout yakni main, kemudian layout:width dan layout:height yang berfungsi untuk mengatur lebar dan panjang dari aplikasi yang pada kasus ini diatur agar match_parent atau memaksimalkan luas layar yang tersedia, ditambahkan padding yang berfungsi memberi jarak bagian tepi ke dalam layar sebanyak 32dp, serta ada context yang memberitahukan komputer jika file xml ini terhubung dengan MainActivity.

Komponen pertama yang diatur pada file ini adalah komponen TextView yang pada baris xml-nya mengatur id sebagai identitas dari komponen yaitu titleText, kemudian mengatur lebar dan panjang dari komponen agar bernilai wrap_content atau menyesuaikan ukuran dari isinya, serta mengambil sumber teks dari resource string calculate_tip. Sementara sisa baris xml dari komponen ini merupakan baris yang mengatur constraint atau posisinya, di mana titleText diminta untuk ditarik ke parent atau menempel pada sisi pinggir atas dan kiri layar. Komponen kedua adalah TextInputLayout yang diberi id billInputLayout, diatur lebarnya 0dp agar mengikuti tarikan constraint dan panjangnya wrap_content, lalu diberikan margin dengan komponen di atasnya sebanyak 16dp. Komponen ini diberikan teks petunjuk dari string bill_amount, menampilkan ikon awalan dari drawable money, serta diatur constraint posisinya agar berada tepat di bawah titleText dan sisi kanannya ditarik ke kanan layar serta sisi kirinya ditarik ke kiri layar. Di dalam TextInputLayout ini membungkus sebuah TextInputEditText dengan id billEditText yang lebarnya ditarik match_parent untuk memaksimalkan luas kontainernya dan tipe inputnya diatur numberDecimal agar memunculkan keyboard khusus angka desimal.

Komponen selanjutnya adalah TextInputLayout yang diberi id tipDropDownLayout, di mana lebarnya juga diatur 0dp dan panjangnya wrap_content,

serta diberikan margin dengan komponen di atasnya sebanyak 16dp. Pada kasus ini komponen diberikan style khusus agar berbentuk seperti menu dropdown (ExposedDropdownMenu), diberikan teks petunjuk dari string tip_percentage, ikon awalan dari drawable percent, dan diatur posisinya agar menempel tepat di bawah billInputLayout sementara sisi kanan dan kirinya ditarik ke parent atau sisi layar. Sama seperti komponen input sebelumnya, komponen ini juga membungkus komponen lain di dalamnya yakni AutoCompleteTextView dengan id tipDropDownText yang lebarnya diatur match_parent dan inputType-nya diatur none agar pengguna tidak bisa mengetik secara manual melainkan hanya bisa memilih opsi yang disediakan oleh menu dropdown.

Selanjutnya ada komponen SwitchMaterial yang pertama-tama diberi id sebagai roundUpSwitch, kemudian diatur panjang dan lebarnya agar wrap_content atau menyesuaikan dengan ukuran tombol itu sendiri. Sisa baris xmlnya mengatur constraint atau posisi dari tombol switch di mana pada kasus ini diatur agar sisi atasnya ditarik ke bagian bawah tipDropDownLayout sementara sisi kanannya ditarik ke parent atau sisi pinggir kanan layar sehingga otomatis posisinya akan melekat di sebelah kanan. Mendampingi switch tersebut, terdapat komponen TextView yang diberi id roundUpText, lebarnya diatur 0dp dan panjangnya wrap_content, diberikan teks dari string round_up_tip dengan gaya huruf ditebalkan (bold) serta ukuran teks 15sp. Sisa baris xmlnya mengatur constraint posisi yang diatur agar sisi kirinya ditarik ke parent atau pinggir kiri layar, sisi kanannya ditarik agar menempel pada sebelah kiri komponen roundUpSwitch, dan bagian atas serta bawahnya diatur untuk ditarik sejajar dengan komponen roundUpSwitch.

Terakhir, terdapat komponen TextView yang berfungsi untuk menampilkan hasil akhir. Pada baris pertamanya diberi id tipResultText, diatur panjang dan lebarnya agar mengikuti isi teks yakni wrap_content, diberikan jarak margin dengan komponen di atasnya sebanyak 16dp, serta diberikan gaya huruf tebal (bold) dengan ukuran teks yang besar yakni 30sp. Sementara untuk sisa baris xmlnya diatur posisinya agar constraint sisi kirinya ditarik ke parent atau pinggir kiri layar, sementara sisi atasnya diatur agar ditarik ke bagian bawah dari komponen roundUpSwitch.

SOAL 2

Jelaskan perbedaan dari implementasi XML dan Jetpack Compose beserta kelebihan dan kekurangan dari masing-masing implementasi.

A. Pembahasan

Perbedaan antara XML dan Jetpack Compose terdapat pada bagaimana keduanya memperbarui tampilan dari view, untuk Compose view langsung dituliskan pada MainActivity.kt dan dideteksi perubahan nilainya misalnya pada TextField menggunakan fungsi onValueChange serta parameter 'callback' yang akan meneruskan nilai baru dari view ke variabel state sehingga secara otomatis memicu pembaruan nilai yang akan ditampilkan pada UI. Sementara untuk XML pada MainActivity.kt perlu untuk memanggil id dari view XML yang dimaksud terlebih dahulu barulah untuk mendeteksi perubahan nilainya menggunakan fungsi yang bertugas sebagai listener seperti doAfterTextChanged misalnya. Barulah digunakan parameter 'callback' yang akan meneruskan nilai baru dari view ke function yang bertugas untuk memicu proses perhitungan dan pembaruan tampilan, sehingga nilai yang ditampilkan pada view akan diperbarui.

Selain itu pada Compose untuk menyusun komponennya dapat menggunakan fungsi seperti Row() untuk menyusun komponennya secara horizontal, dan Column() jika ingin komponen tersusun secara vertikal yang bekerja dengan mengurutkan komponen sesuai dengan urutan penulisan. Sementara untuk XML yang pada kasus ini menggunakan ConstraintLayout maka urutan penulisan komponen view sebenarnya tidaklah penting karena posisi antar komponen ditentukan menggunakan Constraint. Yang dimaksud dengan Constraint adalah seperti mengikat sisi kiri (start), sisi kanan (end), sisi atas(top), dan sisi bawah(Bottom) dari komponen view pada parent atau sisi layar dari handphone, ataupun komponen view lainnya menggunakan id dari komponen tersebut.

Adapun kelebihan dari implementasi menggunakan compose adalah pengaturan posisi dari komponennya pada tampilan di kasus ini lebih simpel dan mudah dipahami karena penyusunan menggunakan fungsi bawaan dan menggunakan prinsip urutan

penulisan. Compose juga menggunakan state untuk memperbarui nilai dari tampilan sehingga kodenya lebih ringkas dibandingkan dengan kode XML dalam kasus memperbarui tampilan, meskipun secara keseluruhan baris kode XML pada projek ini lebih sedikit dibandingkan Compose. Namun dari pengalaman mengerjakan projek ini, Compose juga memiliki kekurangan yakni sulitnya untuk memahami konsep seperti penggunaan state untuk memperbarui tampilan, serta fungsi-fungsi bawaan yang digunakan, dibandingkan memahami alur kerja dari XML yang pada kasus ini lebih simpel.

Sementara kelebihan dari implementasi menggunakan XML adalah pemisahan pengaturan tampilan dan pengaturan logika yang membuat pengerjaan menjadi terasa lebih mudah karena fokus pengerjaan menjadi lebih jelas. Namun kekurangannya adalah kompleksnya proses pengaturan posisi dari view pada tampilan UI karena menggunakan ConstraintLayout, serta proses seperti pembaruan tampilan yang dilakukan secara manual sehingga lebih rentan akan kesalahan ketik.

Soal Essay Additional

1. Apa saja fitur yang menjadi daya jual pada Kotlin? (contoh: null safety, chain, dll)**
2. Berikan 1 test case terkait penggunaan fitur unggulan yang ada pada kotlin (contoh: penggunaan null safety)

A. Pembahasan Soal Additional

1. Beberapa fitur yang menjadi daya tarik dari Kotlin diantaranya adalah:

- Type inference

Pada buku Kotlin in Action, dijelaskan jika Kotlin merupakan bahasa yang bertipe statically typed, di mana tipe data dari variabel harus diketahui saat proses compile berjalan, sehingga jika terjadi kesalahan maka IDE akan mendeteksinya sehingga dapat dilakukan perbaikan secara lebih cepat. Meskipun tipe data dari setiap variabel pada program harus diketahui, Kotlin memiliki fitur bernama type inference, di mana fitur ini memungkinkan Kotlin mengenali tipe data dari suatu variabel berdasarkan tipe data nilainya, tanpa harus menuliskan tipe datanya pada kode, fitur ini membuat penulisan kode menjadi lebih ringkas.

- Data class

Pada Kotlin sebagaimana yang dijelaskan pada buku Kotlin in Action, sebuah class dapat ditambahkan modifier data di bagian depan classnya, maka class tersebut diklasifikasikan menjadi sebuah data class atau class yang tugasnya hanya untuk menyimpan data dan bisa diimplementasi tanpa mengisikan bagian body. Adapun penggunaan data class juga memberikan keuntungan seperti secara otomatis melakukan override pada method-method standard yang biasanya dibutuhkan seperti copy() untuk melakukan duplikasi dan merubah nilai pada variabel yang bertipe val, kemudian equals(), hashCode(), dan toString().

- Interopability With Java

Pada buku Kotlin in Action dijelaskan jika Kotlin dapat berkolaborasi dengan Java seperti memanggil class Java pada projek Kotlin, atau menggunakan library dari Java. Yang mana kemampuan kolaborasi tersebut disebut sebagai interopabillity.

- Lambda Expression

Pada buku Kotlin in Action, dijelaskan jika pada Kotlin terdapat Lambda atau sebuah function yang mampu membuat kode menjadi ringkas, bahkan bisa disimpan pada sebuah variabel, bentuk syntaxnya adalah { parameter -> body }, di mana parameter bisa diisikan dengan sebuah variabel misalnya, barulah pada body dituliskan logika menggunakan parameter yang sudah ditentukan.

- Null Safety

Pada Kotlin sebagaimana dijelaskan pada buku Kotlin Essentials, memiliki fitur bernama null safety, di mana variabel pada Kotlin secara dasar tidak boleh bernilai null, hal ini untuk menghindari NullPointerException atau error yang terjadi karena nilai dari suatu variabel adalah null. Jikapun dirasa perlu dan aman bagi variabel untuk bisa bernilai null maka perlu untuk menggunakan simbol '?', meskipun tidak disarankan.

- Elvis Operator

Untuk berjaga-jaga pada Kotlin terdapat Elvis Operator (?:) yang berdasarkan buku Kotlin Essentials, berfungsi untuk mengembalikan nilai yang dituliskan di bagian kanannya apabila nilai dari variabel ataupun hasil konversi seperti .toDoubleOrNull () misalnya yang dituliskan pada bagian kiri bernilai null. Fitur ini berguna untuk berjaga-jaga apabila pada kode terdapat penggunaan simbol '?' ataupun hal lain seperti input kosong, terutama pada variabel yang nilainya akan digunakan untuk perhitungan.

- Unit return type

Pada buku Kotlin Essentials, dijelaskan bahwa pada Kotlin terdapat Unit return type atau bisa dikatakan sebagai placeholder apabila sebuah function tidak mengembalikan nilai apapun.

2. Test case salah satu fitur dari Kotlin pada pengerjaan modul ini adalah penggunaan Unit Return Type pada parameter callback yang bertugas untuk meneruskan nilai input dari pengguna ke variabel state untuk nantinya memicu perubahan tampilan UI. Adapun kodenya sebagai berikut:

Pertama-tama di function billField itu sendiri dibuat parameter sebagai berikut :

```
@Composable
fun BillField (
    modifier: Modifier = Modifier,
    value : String,
    onValueChange: (String) -> Unit
)
```

Jadi ketika nanti dipanggil ke function yang tugasnya mendeteksi input dan memicu perhitungan yakni TipCalculatorApp, ketika BillField dipanggil ia harus memiliki value yang nilainya itu String, pada kasus ini menyimpan hasil input dari pengguna dari variabel amountInput yang diinisialisasi pada awal function TipCalculatorApp. Dan jika terjadi perubahan maka onValueChangenya harus diisikan dengan fungsi lambda dengan di dalamnya diinisialisasi variabel baru untuk menerima input baru dari pengguna, kemudian nilainya diteruskan pada amountInput yang otomatis juga memperbarui value, dan karena disini digunakan Unit return type maka onValueChange hanya menjalankan proses penerusan nilai input baru tanpa harus mengembalikan nilai apapun. Adapun syntaxnya adalah sebagai berikut

```
fun TipCalculatorApp(modifier: Modifier= Modifier){
    val amountInput = remember { mutableStateOf("") }
    ...
        Column(
        ...
    ) {
        ...

        BillField(
            value = amountInput.value,
            onValueChange = {newValue -> amountInput.value
= newValue}
        )
```

```
        ...
    }
}
```

Nantinya setelah melalui proses yang dijelaskan sebelumnya barulah nilai kedua parameter diteruskan dan akan digunakan kembali pada isi dari billfield dengan syntax

```
@Composable
fun BillField (
    modifier: Modifier = Modifier,
    value : String,
    onValueChange: (String) -> Unit
){
    TextField(
        value = value,
        ...
        onValueChange = onValueChange,
        ...
    )
}
```

TAUTAN GIT

https://github.com/HamkaArifani/Praktikum-Pemrograman-Mobile

DAFTAR PUSTAKA

[1] S. Isakova, R. Elizarov, S. Aigner, dan D. Jemerov, *Kotlin in Action*, edisi ke-2, MEAP v09. Shelter Island, NY, USA: Manning Publications, 2023.
[2] M. Moskała, *Kotlin Essentials*. Leanpub, 2023.